

Presentación breve del lenguaje Scala:

Scala es un lenguaje de programación multiparadigma que combina características de la programación orientada a objetos y la programación funcional. Fue diseñado por Martin Odersky y se ejecuta sobre la máquina virtual de java(JVM), lo que permite aprovechar el ecosistema de Java y sus bibliotecas. Su objetivo es ofrecer un lenguaje expresivo, conciso y seguro para el desarrollo de aplicaciones de distintos tamaños.

Scala se utiliza principalmente en áreas de Big Data, sistemas distribuidos, desarrollo backend y servicios financieros. La fundación Scala destaca que el lenguaje es utilizado para operar aplicaciones distribuidas, pipelines de datos y sistemas críticos en sectores regulados como finanzas y servicios públicos.

```
1  /* funcion: transicion */
2  def transicion(colorActual: String, colorACambiar: String): List[String] = { // declare la funcion, como parametro
3    (colorActual, colorACambiar) match { // funciona como un cond, donde los 2 parametros se comparan en cada caso
4      case ("en-rojo", "en-rojo-intermitente") => List(colorActual, "cambiar-a-rojo-intermitente")
5      // caso 1. => compara los parametros string, cumple con el caso muestra una lista ("en-rojo" "en-rojo-intermitente")
6      case ("en-rojo-intermitente", "en-verde") => List(colorActual, "cambiar-a-verde")
7      // caso 2. => compara los parametros string, cumple con el caso muestra una lista ("en-rojo-intermitente") ("en-verde")
8      case ("en-verde", "en-verde-intermitente") => List(colorActual, "cambiar-a-verde-intermitente")
9      // caso 3. => compara los parametros string, cumple con el caso muestra una lista ("en-verde") ("en-verde-intermitente")
10     case ("en-verde-intermitente", "en-amarillo") => List(colorActual, "cambiar-a-amarillo")
11     // caso 4. => compara los parametros string, cumple con el caso muestra una lista ("en-verde-intermitente") ("en-amarillo")
12     case ("en-amarillo", "en-amarillo-intermitente") => List(colorActual, "cambiar-a-amarillo-intermitente")
13     // caso 5. => compara los parametros string, cumple con el caso muestra una lista ("en-amarillo") ("en-amarillo-intermitente")
14     case ("en-amarillo-intermitente", "en-rojo") => List(colorActual, "cambiar-a-rojo")
15     // caso 6. => compara los parametros string, cumple con el caso muestra una lista ("en-verde") ("en-verde-intermitente")
16     case _ => List(colorActual, "accion-por-defecto")
17     // caso 7. => indica que si no se cumple con las condiciones anteriores, este devuelve el color actual y accion
18   }
19 }
20 // actualizado con intermitentes
```

Función de transición implementada en Scala

La función transición recibe como parámetros colorActual y ColorACambiar(ambos declarados como string) y devuelve una lista string. Dentro de la función utilizamos un match el cual recibirá nuestros parámetros, los cuales en cada caso serán comparados con los colores del semáforo(los cuales deben cumplir con el ciclo rojo→verde→amarillo), si cumple en algún caso se ejecutará y retornará la lista de ese caso, si no cumple con el ciclo del semáforo se retornará una lista con el color actual y el string "accion-por-defecto"

```
43
44 /*funcion timer*/
45 def timer(timestamp: Float): String = { //funcion timer que recibe el parametro timestamp
46   (timestamp % 225.0f) match { //declaro el match con el parametro timestamp
47     // Aplicamos el módulo 225.0f para limpiar los ciclos completos, siempre va a valer entre 0 y 224
48     case time if time >= 0.0f && time <= 89.0f => "en-rojo"
49     // caso 1. => comparamos si esta dentro del rango para ser rojo
50     case time if time >= 90.0f && time <= 92.0f => "en-rojo-intermitente"
51     // caso 2. => comparamos si esta dentro del rango para ser rojo-intermitente
52     case time if time >= 93.0f && time <= 112.0f => "en-verde"
53     // caso 3. => comparamos si esta dentro del rango para ser verde
54     case time if time >= 113.0f && time <= 115.0f => "en-verde-intermitente"
55     // caso 4. => comparamos si esta dentro del rango para ser verde-intermitente
56     case time if time >= 116.0f && time <= 121.0f => "en-amarillo"
57     // caso 5. => comparamos si esta dentro del rango para ser amarillo
58     case time if time >= 122.0f && time <= 124.0f => "en-amarillo-intermitente"
59     // caso 6. => comparamos si esta dentro del rango para ser amarillo-intermitente
60   }
61 }
```

Función de timer implementada en Scala

La función timer recibe como parametro timestap (el cual se declara como float) y retorna un string.

Dentro de la función utilizamos un match el cual recibe timestap % 225, es decir que el match utilizaria el resto de la división entre timestap y 225, donde en cada caso se vera si ese resto , el cual en los caso se los nombra como time cumple con algún rango determinado de los casos, si es asi se ejecuta y se muestra el string.